

R pour jeunes économistes

Manipuler des données cartographique sur R

J.-B. Guiffard

22 mai 2026

Données spatiales et cartes avec R

Objet de la séance

Cette séance est consacrée à l'utilisation de R pour manipuler, analyser et représenter des données spatiales.

L'objectif est de comprendre comment passer :

données géographiques $\rightarrow$ base spatiale $\rightarrow$ carte $\rightarrow$ variable d'analyse

Nous verrons comment utiliser R pour :

- lire des fichiers cartographiques ;
- manipuler des objets spatiaux ;
- produire des cartes ;
- joindre des données statistiques à des géométries ;
- calculer des distances et des intersections ;
- introduire les données raster.

Pourquoi spatialiser les données ?

De nombreuses questions en économie appliquée ont une dimension spatiale.

Exemples :

- accès aux infrastructures ;
- exposition au climat ou à la pollution ;
- distance aux marchés, aux routes ou aux services publics ;
- inégalités territoriales ;
- localisation des entreprises, écoles ou ménages ;
- diffusion spatiale d'un traitement ou d'une politique publique.

⇒ Spatialiser les données permet de relier des phénomènes économiques à des lieux, des distances et des territoires.

Plan de la séance

- 1 Comprendre les principaux types de données spatiales
- 2 Lire et manipuler des données vectorielles avec sf
- 3 Produire des cartes avec ggplot2, tmap et mapview
- 4 Joindre des données socio-économiques à des données géographiques
- 5 Manipuler des points, des distances et des rasters

Pourquoi les données spatiales en économie ?

Pourquoi spatialiser les données ?

De nombreuses questions en économie appliquée ont une dimension spatiale.

Les phénomènes économiques ne se distribuent pas de manière uniforme dans l'espace :

- les infrastructures sont localisées ;
- les ménages et les entreprises sont situés à des endroits précis ;
- les politiques publiques peuvent cibler certains territoires ;
- les conditions climatiques varient selon les lieux ;
- l'accès aux marchés, aux écoles ou aux services publics dépend de la distance.

Quelques exemples de questions de recherche

Les données spatiales peuvent être utiles dans de nombreux champs de l'économie.

- Quel est l'effet de l'accès à une route sur l'activité économique locale ?
- Les écoles connectées à Internet obtiennent-elles de meilleurs résultats ?
- Comment l'exposition à la sécheresse affecte-t-elle les revenus agricoles ?
- Les entreprises proches d'une infrastructure numérique deviennent-elles plus productives ?
- Les ménages éloignés des services publics sont-ils moins bien couverts ?
- Comment les politiques publiques se diffusent-elles entre territoires voisins ?

Trois usages des données spatiales

Dans un projet empirique, les données spatiales peuvent servir à plusieurs choses.

- 1 **Décrire** : représenter la distribution spatiale d'un phénomène.
- 2 **Construire des variables** : distance, exposition, accessibilité, appartenance à une zone.
- 3 **Analyser** : étudier les inégalités territoriales, les effets de voisinage ou l'impact d'une politique localisée.

Exemples :

- distance à une route, un port ou une capitale ;
- exposition à une température ou une sécheresse ;
- nombre d'écoles dans une commune ;
- appartenance à une zone traitée.

Exemple 1 : infrastructures et développement

Supposons que l'on étudie l'effet d'une nouvelle route, d'un câble Internet ou d'une ligne électrique.

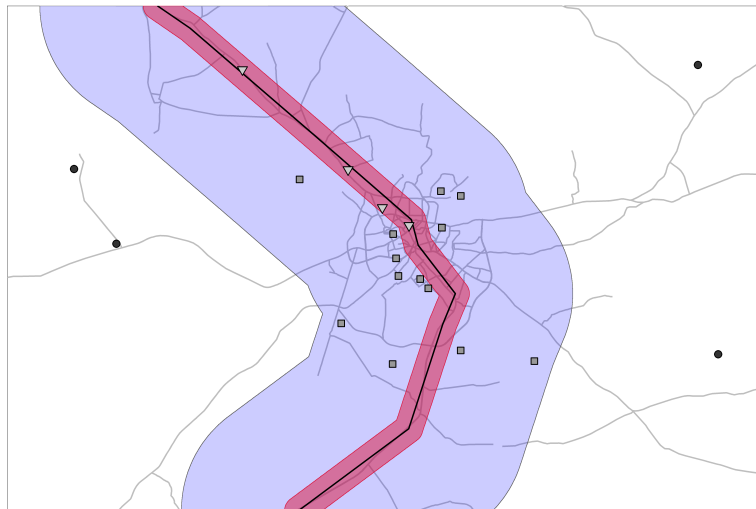
Les données spatiales permettent de construire plusieurs variables :

- distance à l'infrastructure ;
- présence de l'infrastructure dans une zone ;
- date d'arrivée de l'infrastructure ;
- nombre d'infrastructures autour d'un lieu ;
- exposition différenciée selon la proximité.

Ces variables peuvent ensuite être utilisées dans une analyse économétrique.

Ex : Comparer les zones proches et éloignées d'une infrastructure avant et après son déploiement.

Exemple 1 : infrastructures et développement



connected ▼ 1-Connected ■ 2-Un-connected ● 3-Out of sample

Exemple 2 : climat, agriculture et revenus

Les données climatiques sont souvent disponibles sous forme de grilles spatiales.

Chaque pixel peut contenir une information :

- température ;
- précipitations ;
- sécheresse ;
- humidité des sols ;
- végétation ;
- altitude.

Exemple 2

On peut ensuite associer ces informations à des unités économiques :

- ménages ;
- exploitations agricoles ;
- communes ;
- régions ;
- pays.

Question empirique : Comment l'exposition à un choc climatique affecte-t-elle les revenus, les migrations ou la production agricole ?

Exemple 2



Exemple 3 : services publics et accessibilité

L'accès aux services publics dépend souvent de la localisation.

On peut étudier par exemple :

- la distance à l'école la plus proche ;
- la distance à un centre de santé ;
- le temps d'accès à une route ;
- la couverture d'un réseau mobile ;
- la proximité d'une administration publique.

Ces mesures permettent de construire des indicateurs d'accessibilité.

Exemple 4 : politiques publiques localisées

Certaines politiques publiques ne s'appliquent pas partout de la même manière.

Elles peuvent cibler :

- certaines communes ;
- certaines écoles ;
- certaines régions ;
- certaines zones frontalières ;
- certaines zones exposées à un risque.

Exemple 4

Les données spatiales permettent alors de définir :

- les zones traitées ;
- les zones de comparaison ;
- les distances aux frontières du traitement ;
- les unités exposées indirectement.

La localisation peut aider à définir un traitement, une exposition ou un groupe de contrôle.

Exemple 4



Carte descriptive ou variable empirique ?

Une carte peut avoir deux fonctions différentes.

1 Illustrer un phénomène

- carte du revenu par région ;
- carte des émissions de CO₂ par pays ;
- carte de la densité de population.

2 Construire une variable d'analyse

- distance à une infrastructure ;
- exposition à un choc climatique ;
- nombre d'écoles dans une zone ;
- appartenance à une zone traitée.

Ce que nous allons apprendre à faire

Dans cette séance, nous allons voir comment utiliser R pour :

- lire une carte sous forme de fichier spatial ;
- comprendre la structure d'un objet spatial ;
- vérifier un système de coordonnées ;
- joindre une base statistique à une carte ;
- produire une carte thématique ;
- manipuler des points et des polygones ;
- calculer des distances ou des intersections ;
- introduire les données raster.

Comprendre les objets spatiaux

Qu'est-ce qu'une donnée spatiale ?

Une donnée spatiale est une donnée associée à une localisation.

Elle contient généralement deux types d'information :

- une information **attributaire** : nom, population, revenu, émission, statut ;
- une information **géographique** : coordonnées, forme, position, surface.

Exemples :

- une école avec sa latitude et sa longitude ;
- une commune avec ses frontières ;
- une route avec son tracé ;
- une grille climatique avec une température par pixel.

Une donnée spatiale est une donnée classique à laquelle on ajoute une dimension géographique.

Les grandes familles de données spatiales

On distingue principalement deux grands types de données spatiales.

1 Données vectorielles

- points ;
- lignes ;
- polygones.

2 Données raster

- grille de pixels ;
- une valeur associée à chaque cellule ;
- utile pour les phénomènes continus.

Les données vectorielles

Les données vectorielles représentent des objets géographiques précis.

Elles peuvent prendre plusieurs formes :

- **points** : écoles, entreprises, ménages, antennes, centrales ;
- **lignes** : routes, chemins de fer, rivières, câbles ;
- **polygones** : pays, régions, communes, zones administratives.



Les données vectorielles

Exemples en économie appliquée :

- distance d'un ménage à une route ;
- nombre d'écoles dans une commune ;
- exposition d'une entreprise à une zone traitée.

Les données raster

Les données raster sont organisées sous forme de grille.

Chaque cellule ou pixel contient une valeur.

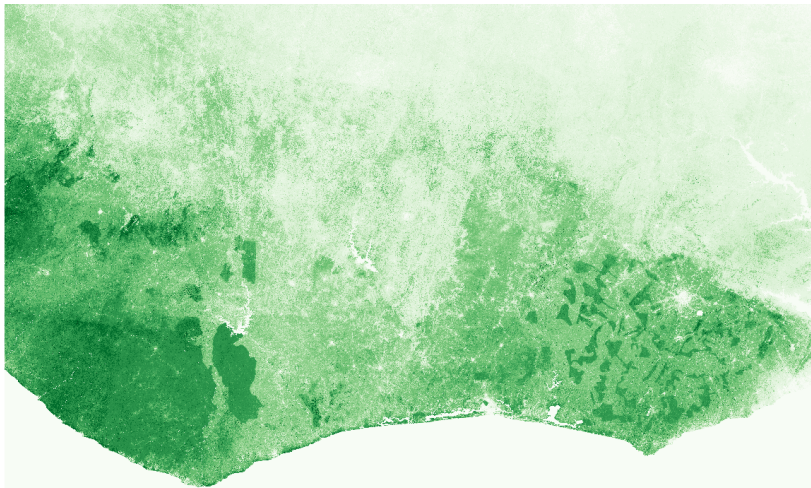
Exemples :

- température ;
- précipitations ;
- altitude ;
- luminosité nocturne ;
- occupation des sols ;
- indice de végétation.

Ces données sont très utilisées pour mesurer des phénomènes continus dans l'espace.

Ex : On peut calculer la température moyenne observée dans chaque commune à partir d'un raster climatique.

Les données raster



Points, polygones et rasters : exemples

Une même question de recherche peut combiner plusieurs types de données.

Exemple : étudier l'effet des infrastructures scolaires sur les résultats éducatifs.

- **Points** : localisation des écoles ;
- **Polygones** : communes ou districts scolaires ;
- **Rasters** : densité de population ou accessibilité routière.

On peut ensuite construire des variables :

- nombre d'écoles par commune ;
- distance à l'école la plus proche ;
- population exposée dans un rayon donné.

Coordonnées géographiques

Pour localiser un point sur la Terre, on utilise souvent deux coordonnées :

longitude, latitude

- la **longitude** indique la position est-ouest ;
- la **latitude** indique la position nord-sud.

Exemple :

Paris $\approx 2.35^\circ, 48.85^\circ$

Attention : Les coordonnées en longitude-latitude sont exprimées en degrés, pas en mètres.

CRS : système de coordonnées

Un CRS indique comment les coordonnées géographiques doivent être interprétées. CRS signifie Coordinate Reference System

Il définit notamment :

- le système de référence utilisé ;
- l'unité des coordonnées ;
- la manière de représenter la Terre ;
- la projection éventuelle utilisée.

Exemples fréquents :

- EPSG:4326 : coordonnées longitude-latitude, souvent utilisé par le GPS ;
- EPSG:3857 : projection souvent utilisée pour les cartes web ;
- EPSG:2154 : Lambert-93, souvent utilisé pour la France.

Avant toute opération spatiale, vérifier le CRS des données.

Pourquoi les projections comptent ?

La Terre est une surface courbe.

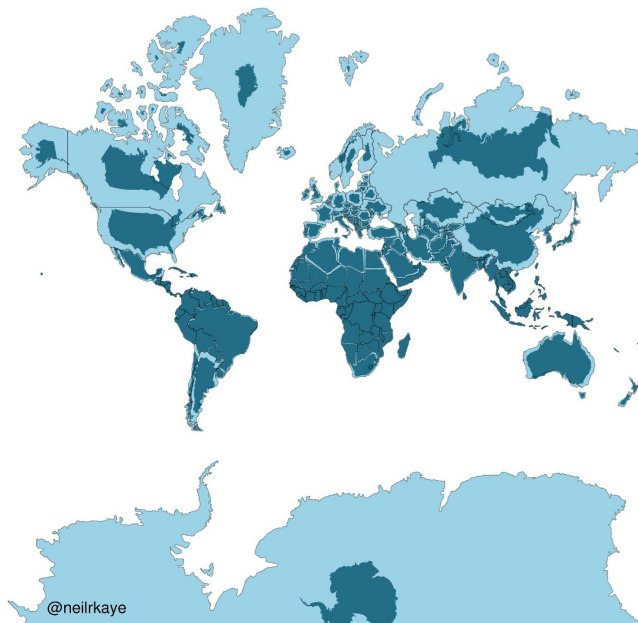
Une carte est une représentation plane de cette surface.

Passer d'un globe à une carte implique donc toujours des déformations :

- des surfaces ;
- des formes ;
- des distances ;
- des angles.

Aucune projection n'est parfaite : le choix dépend de l'usage.

Le bon système de projection dépend de ce que l'on veut faire : afficher, mesurer une distance, calculer une surface ou comparer des territoires.



Projection et calcul de distance

Les calculs de distance sont sensibles au système de coordonnées.

Si les données sont en longitude-latitude :

les coordonnées sont en degrés

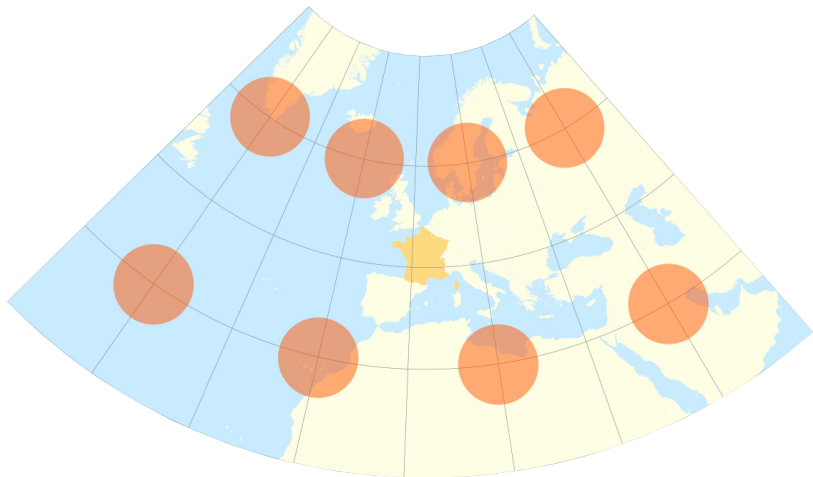
Si les données sont projetées :

les coordonnées peuvent être en mètres

Pour calculer des distances locales, il est souvent préférable d'utiliser une projection adaptée.

Ex : Pour travailler sur la France métropolitaine, on utilise souvent le Lambert-93 : EPSG:2154.

Projection et calcul de distance



Projection et calcul de distance



Formats de fichiers fréquents

Les données spatiales peuvent être stockées dans plusieurs formats.

Pour les données vectorielles :

- `.shp` : shapefile, format très répandu ;
- `.gpkg` : GeoPackage, format plus moderne ;
- `.geojson` : format pratique pour le web.

Pour les données raster :

- `.tif` ou `.tiff` : GeoTIFF ;
- `.nc` : NetCDF, souvent utilisé pour les données climatiques.

Bonne pratique : Pour les nouveaux projets, le format GeoPackage est souvent plus pratique qu'un shapefile.

L'objet sf dans R

Dans R, le package sf permet de manipuler des données vectorielles.

Un objet sf ressemble à un tableau de données classique.

La différence principale est qu'il contient une colonne spéciale : **geometry**

Cette colonne contient les coordonnées ou les formes géographiques.

L'objet sf

TYPE_1	NL_NAME_1	VARNAME_1	geometry
'norate	NA	Al Arylānah L'Ariana Tunis Ariana Al Aryānah	list(list(c(10.2109699249268, 10.2109699249268, 10.2106895...
'norate	NA	Bājah Béja	list(list(c(9.08780288696289, 9.08878231048584, 9.09033107...
'norate	NA	Bin 'Arūs Ben Arous Tunis Ben Arous	list(list(c(10.2948608398438, 10.2948608398438, 10.2962512...
'norate	NA	BanzartBanzart Bensert Binzart Biserta Bizerta	list(list(c(10.2334718704225, 10.2334718704225, 10.2337503...
'norate	NA	Qābis Gabās	list(list(c(9.67866897583008, 9.69477558135992, 9.69743347...

Pourquoi sf est pratique ?

Comme un objet sf est proche d'un tableau classique, on peut souvent utiliser les outils du tidyverse.

Par exemple, on peut :

- sélectionner des variables avec `select()` ;
- filtrer des observations avec `filter()` ;
- créer des variables avec `mutate()` ;
- faire des jointures avec `left_join()` ;
- agréger avec `group_by()` et `summarise()`.

Les opérations spatiales de base

Les données spatiales permettent de réaliser des opérations spécifiques.

Exemples :

- calculer une distance entre deux objets ;
- savoir si un point se trouve dans un polygone ;
- créer une zone tampon autour d'un lieu ;
- agréger plusieurs polygones ;
- calculer une surface ;
- extraire une valeur raster dans une zone.

Les erreurs classiques avec les objets spatiaux

Quelques erreurs sont très fréquentes.

- Calculer une distance alors que les coordonnées sont en degrés.
- Joindre deux bases avec des codes géographiques incompatibles.
- Mélanger des objets avec des CRS différents.
- Oublier les valeurs manquantes après une jointure.
- Interpréter une carte sans regarder les classes de couleur.
- Confondre une carte descriptive avec une relation causale.

Lire et manipuler des données avec sf

Charger les packages utiles

On commence par charger les packages nécessaires.

```
library(tidyverse)  
library(sf)
```

Le package `sf` permet de lire, manipuler et sauvegarder des données vectorielles.

Le package `tidyverse` permet d'utiliser les fonctions habituelles de manipulation de données.

Bonne pratique : Toujours charger les packages au début du script pour rendre le code plus lisible.

Importer une couche géographique

On importe une couche spatiale avec la fonction `st_read()`.

```
world_map <- st_read("data/raw/world-administrative-boundaries.shp")
```

Cette commande crée un objet appelé `world_map`.

L'objet contient :

- des variables attributaires ;
- une colonne géographique ;
- le système de coordonnées de la couche.

`st_read()` est l'équivalent spatial de `read_csv()`.

Inspecter un objet sf

Un objet sf peut être inspecté comme une base de données classique.

```
glimpse(world_map)
```

```
names(world_map)
```

```
head(world_map)
```

On peut alors identifier :

- les variables disponibles ;
- le nombre d'observations ;
- les codes géographiques ;
- la colonne de géométrie.

Avant toute carte, toujours regarder la structure de l'objet spatial.

Afficher rapidement la géométrie

On peut afficher uniquement la géométrie d'un objet sf.

```
plot(st_geometry(world_map))
```

Cette commande permet de vérifier rapidement :

- si la couche s'affiche correctement ;
- si les formes géographiques semblent cohérentes ;
- si l'étendue spatiale correspond à ce que l'on attend.

Bonne pratique : Un premier plot() permet souvent de repérer rapidement un problème de géométrie ou de projection.

Vérifier le système de coordonnées

Chaque couche spatiale possède un système de coordonnées.

On peut le vérifier avec :

```
st_crs(world_map)
```

Cette commande indique notamment :

- le code EPSG ;
- le nom du système de coordonnées ;
- l'unité utilisée ;
- la projection éventuelle.

Avant de calculer une distance ou une surface, toujours vérifier le CRS.

Sélectionner des variables

Comme avec une base classique, on peut sélectionner certaines colonnes.

```
world_small <- world_map %>%  
  select(name, iso3, continent, geometry)
```

La colonne geometry contient les formes géographiques.

Même si elle n'est pas explicitement sélectionnée, sf conserve souvent la géométrie active.

Filtrer des objets géographiques

On peut filtrer des pays, régions ou communes avec `filter()`.

```
europe_map <- world_map %>%  
  filter(continent == "Europe")
```

On peut ensuite afficher le résultat.

```
plot(st_geometry(europe_map))
```

Ex : Ici, on crée une nouvelle couche contenant uniquement les pays européens.

Créer de nouvelles variables

On peut créer de nouvelles variables comme dans une base classique.

```
world_map <- world_map %>%  
  mutate(  
    is_europe = continent == "Europe"  
  )
```

Cette variable peut ensuite être utilisée :

- pour filtrer la base ;
- pour colorer une carte ;
- pour définir un groupe d'analyse.

Supprimer temporairement la géométrie

Parfois, on souhaite travailler uniquement sur la table attributaire.

On peut retirer la géométrie avec :

```
world_table <- world_map %>%  
  st_drop_geometry()
```

Cela permet de produire des tableaux, des statistiques ou des vérifications sans conserver les géométries.

Exemple :

```
world_table %>%  
  count(continent)
```

Bonne pratique : Utiliser `st_drop_geometry()` lorsque la géométrie n'est pas nécessaire.

Transformer une projection

On peut changer le système de coordonnées avec `st_transform()`.

```
world_map_3857 <- world_map %>%  
  st_transform(3857)
```

Pour la France métropolitaine, on utilise souvent le Lambert-93 :

```
france_map_2154 <- world_map %>%  
  filter(iso3 == "FRA") %>%  
  st_transform(2154)
```

`st_transform()` ne change pas la forme réelle des objets : il change leur représentation dans un autre système de coordonnées.

Calculer une surface

Une fois la projection adaptée, on peut calculer des surfaces.

```
france_map_2154 <- france_map_2154 %>%  
  mutate(  
    area_km2 = as.numeric(st_area(geometry)) / 1e6  
  )
```

La fonction `st_area()` calcule la surface des géométries.

On divise par `1e6` pour passer de mètres carrés à kilomètres carrés.

→ Pour calculer une surface, utiliser une projection adaptée et vérifier l'unité.

Calculer un centroïde

Un centroïde est un point qui résume la position d'un polygone.

```
world_centroids <- world_map %>%  
  st_centroid()
```

Les centroïdes sont utiles pour :

- placer des labels ;
- calculer des distances approximatives ;
- représenter des points à partir de polygones ;
- simplifier certaines opérations spatiales.

Attention : Le centroïde n'est pas toujours situé à l'intérieur du polygone, notamment pour les formes complexes.

Agréger des polygones

On peut agréger plusieurs polygones avec `group_by()` et `summarise()`.

```
continents_map <- world_map %>%  
  group_by(continent) %>%  
  summarise(  
    n_countries = n(),  
    .groups = "drop"  
  )
```

Cette opération permet de passer : pays $\rightarrow$ continents

Avec `sf`, `summarise()` agrège aussi les géométries.

Créer une zone tampon

Une zone tampon, ou *buffer*, correspond à une zone autour d'un objet spatial.

```
france_buffer <- france_map_2154 %>%  
  st_buffer(dist = 100000)
```

Ici, on crée une zone de 100 km autour de la France.

Les buffers sont utiles pour construire des variables d'exposition :

- être situé à moins de 10 km d'une route ;
- être situé autour d'une école ;
- être proche d'une frontière ;
- être exposé à une infrastructure.

Attention : La distance du buffer dépend de l'unité du CRS utilisé.

Sauvegarder une couche spatiale

Après manipulation, on peut sauvegarder une nouvelle couche.

```
st_write(  
  europe_map,  
  "data/clean/europe_map.gpkg",  
  delete_dsn = TRUE  
)
```

Le format .gpkg est pratique car il stocke les données et la géométrie dans un seul fichier.

Bonne pratique : Sauvegarder les couches nettoyées dans un dossier data/clean.

Chaîne simple de traitement

On peut regrouper plusieurs étapes dans une seule chaîne.

```
europa_map <- world_map %>%  
  filter(continent == "Europe") %>%  
  select(name, iso3, continent, geometry) %>%  
  st_transform(3035) %>%  
  mutate(  
    area_km2 = as.numeric(st_area(geometry)) / 1e6  
  )
```

Cette chaîne permet de :

- sélectionner l'Europe ;
- garder quelques variables utiles ;
- changer de projection ;
- calculer une surface.

À retenir

Le package `sf` permet de manipuler simplement les données vectorielles.

Les commandes principales à retenir sont :

- `st_read()` : importer une couche spatiale ;
- `st_crs()` : vérifier le système de coordonnées ;
- `st_transform()` : changer de projection ;
- `st_geometry()` : accéder à la géométrie ;
- `st_drop_geometry()` : retirer la géométrie ;
- `st_area()` : calculer une surface ;
- `st_centroid()` : calculer des centroïdes ;
- `st_buffer()` : créer une zone tampon ;
- `st_write()` : sauvegarder une couche.

Produire une première carte

Produire une première carte

Dans ce bloc, nous allons produire nos premières cartes avec R.

Nous verrons trois approches complémentaires :

- `plot()` : pour vérifier rapidement une couche spatiale ;
- `ggplot2` : pour produire une carte statique propre ;
- `tmap` et `mapview` : pour produire des cartes cartographiques ou interactives.

Une première carte avec plot()

La fonction `plot()` permet de visualiser rapidement un objet spatial.

```
plot(st_geometry(world_map))
```

On peut aussi représenter une variable attributaire.

```
plot(world_map["continent"])
```

`plot()` est très utile pour vérifier rapidement que les données s'affichent correctement.

Une carte avec ggplot2

On peut produire une carte avec ggplot2 en utilisant `geom_sf()`.

```
ggplot(world_map) +  
  geom_sf()
```

`geom_sf()` reconnaît automatiquement la colonne de géométrie de l'objet `sf`.

Colorer une carte par continent

On peut utiliser une variable attributaire pour colorer les polygones.

```
map_continent <- ggplot(world_map) +  
  geom_sf(  
    aes(fill = continent),  
    color = "grey70",  
    size = 0.1  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Carte des continents",  
    fill = "Continent"  
  )  
map_continent
```


Améliorer la lisibilité

Une carte doit rester simple et lisible.

On peut supprimer les axes et simplifier l'apparence.

```
map_continent <- ggplot(world_map) +  
  geom_sf(  
    aes(fill = continent),  
    color = "grey70",  
    size = 0.1  
  ) +  
  theme_minimal() +  
  theme(  
    axis.title = element_blank(),  
    axis.text = element_blank(),  
    panel.grid = element_blank()  
  ) +  
  labs(  
    title = "Carte des continents",  
    fill = "Continent",  
    caption = "Source : données administratives mondiales"
```

Carte qualitative ou quantitative ?

Toutes les variables ne se cartographient pas de la même manière.

On distingue souvent :

- les variables **qualitatives** : continent, région, type de zone ;
- les variables **quantitatives** : population, revenu, émissions, densité.

Exemples :

- continent : variable qualitative ;
- population : variable quantitative ;
- co2_per_capita : variable quantitative.

Une carte quantitative

Pour une variable quantitative, on utilise une échelle continue.

Exemple avec une variable de population si elle est disponible dans la base :

```
ggplot(world_map) +  
  geom_sf(  
    aes(fill = population),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  labs(title = "Population par pays",  
        fill = "Population" )
```

Attention : Les variables très asymétriques peuvent être difficiles à représenter avec une échelle continue.

Gérer les valeurs manquantes

Une carte peut contenir des pays ou zones sans données.

Il faut les rendre visibles.

```
ggplot(world_map) +  
  geom_sf(  
    aes(fill = population),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Population par pays",  
    fill = "Population",  
    caption = "Les zones en gris correspondent aux valeurs manquantes"  
  )
```

Créer des classes

Pour certaines cartes, une variable catégorisée est plus lisible qu'une échelle continue.

```
world_map <- world_map %>%  
  mutate(  
    pop_class = cut(  
      population,  
      breaks = c(0, 1e6, 1e7, 5e7, 1e8, Inf),  
      labels = c(  
        "< 1 million",  
        "1-10 millions",  
        "10-50 millions",  
        "50-100 millions",  
        "> 100 millions"  
      )  
    )  
  )  
)
```

Attention : Le choix des classes influence fortement la lecture de la carte.

Cartographier des classes

Une fois la variable catégorielle créée, on peut la représenter.

```
ggplot(world_map) +  
  geom_sf(  
    aes(fill = pop_class),  
    color = "grey80",  
    size = 0.05  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Population par pays",  
    fill = "Classe de population"  
  )
```

Les classes facilitent la lecture, mais elles simplifient l'information initiale.

Une carte avec tmap

Le package tmap est conçu spécifiquement pour faire des cartes.

```
library(tmap)  
  
tm_shape(world_map) +  
  tm_polygons("continent")
```

tmap est utile pour produire rapidement des cartes cartographiques propres.

ggplot2 est très flexible ; tmap est souvent plus directement pensé pour la cartographie.

Une carte interactive avec mapview

Le package mapview permet de visualiser rapidement une couche sur une carte interactive.

```
library(mapview)
```

```
mapview(world_map)
```

On peut zoomer, se déplacer et explorer les objets spatiaux.

mapview est très utile pour explorer les données, mais moins adapté à une figure finale d'article.

Quel outil utiliser ?

Chaque outil a un usage différent.

- `plot()` : vérifier rapidement les données ;
- `ggplot2` : produire des figures intégrées à un workflow tidyverse ;
- `tmap` : produire des cartes cartographiques rapidement ;
- `mapview` : explorer les données de manière interactive.

Pour un article ou un rapport, on privilégie souvent `ggplot2` ou `tmap`. Pour explorer les données, `mapview` est très pratique.

Les choix qui comptent

Une carte n'est jamais neutre.

Plusieurs choix influencent son interprétation :

- la projection utilisée ;
- la variable représentée ;
- l'échelle de couleur ;
- le choix des classes ;
- la gestion des valeurs manquantes ;
- le niveau géographique retenu ;
- le titre et la légende.

Une carte doit être lue comme un résultat construit, pas comme une simple image des données.

Joindre données statistiques et carte

Joindre données statistiques et carte

Une carte thématique combine généralement deux types de données.

- une couche géographique : pays, régions, communes, districts ;
- une base statistique : population, revenu, émissions, indicateurs socio-économiques.

L'objectif est de joindre ces deux sources pour produire une carte exploitable.

Exemple de question

Nous partons d'une question simple :

Comment les émissions de CO2 par habitant se répartissent-elles dans le monde ?

Pour répondre à cette question, nous avons besoin :

- d'une carte des pays du monde ;
- d'une base statistique sur les émissions de CO2 ;
- d'un identifiant commun pour faire la jointure ;
- d'une variable à cartographier.

Avant une jointure, il faut toujours identifier la clé commune entre les deux bases.

Identifier la clé de jointure

Pour joindre deux bases, il faut une variable commune.

Dans notre exemple :

- la carte contient un code pays : `iso3` ;
- la base CO2 contient un code pays : `iso_code`.

On va donc joindre les deux bases avec :

```
world_co2 <- world_map %>%  
  left_join(  
    co2_2019,  
    by = c("iso3" = "iso_code")  
  )
```

Attention : Une grande partie des erreurs de carte vient de jointures mal faites.

Importer la base statistique

On commence par importer la base CO2.

```
co2_raw <- read_csv("data/raw/owid-co2-data.csv")
```

On garde ensuite l'année 2019 et quelques variables utiles.

```
co2_2019 <- co2_raw %>%  
  filter(year == 2019) %>%  
  select(  
    iso_code, country, year,  
    co2_per_capita, population, gdp  
  )
```

Bonne pratique : Avant de joindre, créer une base statistique simple avec uniquement les variables utiles.

Vérifier la base à joindre

Avant la jointure, on inspecte rapidement la base statistique.

```
glimpse(co2_2019)
```

```
co2_2019 %>%  
  summarise(  
    n_obs = n(),  
    n_countries = n_distinct(iso_code),  
    missing_co2_pc = sum(is.na(co2_per_capita))  
  )
```

On peut aussi vérifier les codes pays manquants.

```
co2_2019 %>%  
  filter(is.na(iso_code)) %>%  
  select(country)
```

Toujours vérifier la base statistique avant de la joindre à une carte.

Réaliser la jointure

On joint la base statistique à la carte avec `left_join()`.

```
world_co2 <- world_map %>%  
  left_join(  
    co2_2019,  
    by = c("iso3" = "iso_code")  
  )
```

Ici, on garde toutes les géométries de la carte et on ajoute les variables CO2 lorsqu'une correspondance existe.

Vérifier le résultat de la jointure

Après une jointure, il faut toujours vérifier ce qui a été apparié ou non.

```
world_co2 %>%  
  summarise(  
    n_polygons = n(),  
    matched_co2 = sum(!is.na(co2_per_capita)),  
    missing_co2 = sum(is.na(co2_per_capita))  
  )
```

On peut aussi afficher les pays sans données CO2.

```
world_co2 %>%  
  filter(is.na(co2_per_capita)) %>%  
  st_drop_geometry() %>%  
  select(name, iso3)
```

Bonne pratique : Une jointure doit toujours être contrôlée : certaines unités peuvent ne pas être appariées.

Comprendre les non-appariements

Les non-appariements peuvent venir de plusieurs sources.

- codes pays différents ;
- territoires présents dans la carte mais absents de la base statistique ;
- agrégats présents dans la base statistique mais absents de la carte ;
- changements de frontières ou de noms ;
- valeurs manquantes pour certaines années.

Exemples fréquents :

- Kosovo ;
- territoires ultramarins ;
- petits États insulaires ;
- régions ou agrégats comme *World, Europe, Asia*.

Un pays non apparié n'est pas forcément une erreur, mais il doit être identifié.

Première carte des émissions de CO2

Une fois la jointure réalisée, on peut cartographier la variable.

```
ggplot(world_co2) +  
  geom_sf(  
    aes(fill = co2_per_capita),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Émissions de CO2 par habitant en 2019",  
    fill = "CO2 par habitant",  
    caption = "Source : Our World in Data"  
  )
```

Les pays en gris correspondent aux pays sans valeur disponible ou non

Rendre la carte plus lisible

Pour une carte thématique, on peut simplifier l'apparence.

```
map_co2_2019 <- ggplot(world_co2) +  
  geom_sf(  
    aes(fill = co2_per_capita),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  theme(  
    axis.title = element_blank(),  
    axis.text = element_blank(),  
    panel.grid = element_blank()  
  ) +  
  labs(  
    title = "Émissions de CO2 par habitant en 2019",
```

Créer des classes de valeurs

Pour certaines variables très asymétriques, une carte en classes peut être plus lisible.

```
world_co2 <- world_co2 %>%  
  mutate(  
    co2_class = cut(  
      co2_per_capita,  
      breaks = c(0, 1, 3, 5, 10, 20, Inf),  
      labels = c(  
        "< 1",  
        "1-3",  
        "3-5",  
        "5-10",  
        "10-20",  
        "> 20"  
      )  
    )  
  )
```

Cartographier les classes

On peut ensuite représenter la variable catégorisée.

```
ggplot(world_co2) +  
  geom_sf(  
    aes(fill = co2_class),  
    color = "grey80",  
    size = 0.05  
  ) +  
  theme_minimal() +  
  theme(  
    axis.title = element_blank(),  
    axis.text = element_blank(),  
    panel.grid = element_blank()  
  ) +  
  labs(  
    title = "Émissions de CO2 par habitant en 2019",  
    fill = "Tonnes par habitant",  
    caption = "Source : Our World in Data"  
  )
```

Comparer deux variables cartographiées

Une fois la jointure faite, on peut cartographier différentes variables.

Par exemple, la population :

```
ggplot(world_co2) +  
  geom_sf(  
    aes(fill = population),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    trans = "log10",  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Population par pays en 2019",  
    fill = "Population",  
    caption = "Source : Our World in Data"  
  )
```


Attention aux échelles

Les cartes peuvent donner des impressions différentes selon la variable représentée.

Exemples :

- émissions totales de CO2 ;
- émissions de CO2 par habitant ;
- PIB total ;
- PIB par habitant ;
- population ;
- densité de population.

Ces variables ne répondent pas aux mêmes questions.

Carte et interprétation

Une carte des émissions de CO2 par habitant permet de visualiser des différences spatiales.

Mais elle ne permet pas directement d'expliquer ces différences.

Pour interpréter la carte, il faut tenir compte :

- du niveau de développement ;
- de la structure productive ;
- de la géographie ;
- des ressources énergétiques ;
- de la population ;
- des choix de mesure.

Une carte décrit une distribution spatiale ; elle ne suffit pas à identifier une relation causale.

Sauvegarder la carte

On peut sauvegarder une carte avec `ggsave()`.

```
ggsave(  
  filename = "outputs/figures/map_co2_2019.png",  
  plot = map_co2_2019,  
  width = 10,  
  height = 6,  
  dpi = 300  
)
```

Cela permet de réutiliser la carte dans :

- un article ;
- un rapport ;
- une présentation ;
- un document RMarkdown.

Bonne pratique : Sauvegarder les figures produites dans un dossier `outputs/figures`.

Points, distances et intersections

Points, distances et intersections

Les données spatiales ne servent pas seulement à produire des cartes.
Elles permettent aussi de construire des variables utiles pour l'analyse empirique.

Dans ce bloc, nous allons voir comment :

- manipuler des données ponctuelles ;
- convertir une table avec coordonnées en objet spatial ;
- projeter des points sur une carte ;
- calculer des distances ;
- compter des points dans des polygones ;
- identifier des intersections spatiales.

Pourquoi travailler avec des points ?

De nombreuses données utilisées en économie appliquée sont localisées par des coordonnées.

Exemples :

- écoles ;
- ménages ;
- entreprises ;
- hôpitaux ;
- antennes téléphoniques ;
- centrales électriques ;
- conflits ou manifestations ;
- infrastructures publiques.

Ces observations peuvent être représentées comme des points.

Dès qu'une observation a une latitude et une longitude, elle peut devenir

Exemple de question empirique

Supposons que l'on dispose de la localisation de centrales électriques.

On peut se poser plusieurs questions :

- Dans quels pays ces centrales sont-elles situées ?
- Combien de centrales se trouvent dans chaque pays ?
- Quelle est la distance entre chaque pays et la centrale la plus proche ?
- Quels territoires sont situés à moins de 50 km d'une centrale ?
- Les zones proches d'une infrastructure ont-elles des trajectoires économiques différentes ?

Ex : Transformer des coordonnées géographiques en variables d'exposition à une infrastructure.

Charger une base de points

On peut lire une couche de points comme n'importe quelle couche spatiale.

```
power_plants <- st_read("data/raw/Global_Power_Plants/Power_Plant
```

On inspecte ensuite l'objet.

```
glimpse(power_plants)
```

```
st_crs(power_plants)
```

```
plot(st_geometry(power_plants))
```

Comme pour les polygones, on vérifie toujours la structure, la géométrie et le CRS.

Créer des points à partir de coordonnées

Parfois, les données ne sont pas encore spatiales.

Elles peuvent simplement contenir deux colonnes :

longitude, latitude

On peut les convertir en objet sf avec `st_as_sf()`.

```
power_plants_sf <- power_plants_df %>%  
  st_as_sf(  
    coords = c("longitude", "latitude"),  
    crs = 4326  
  )
```

Attention : Il faut indiquer le bon CRS : les coordonnées longitude-latitude sont généralement en EPSG:4326.

Afficher des points sur une carte

On peut superposer les points à une carte de polygones.

```
ggplot() +  
  geom_sf(data = world_map, fill = "grey95", color = "grey80") +  
  geom_sf(data = power_plants, size = 0.5, alpha = 0.6) +  
  theme_minimal() +  
  labs(  
    title = "Localisation des centrales électriques"  
  )
```

Les polygones donnent le contexte géographique ; les points représentent les unités localisées.

Filtrer un type de points

Si la base contient plusieurs types d'infrastructures, on peut filtrer les observations.

```
nuclear_plants <- power_plants %>%  
  filter(fuel1 == "Nuclear")
```

Puis afficher uniquement ces points.

```
ggplot() +  
  geom_sf(data = world_map, fill = "grey95", color = "grey80") +  
  geom_sf(data = nuclear_plants, size = 0.8, alpha = 0.7) +  
  theme_minimal() +  
  labs(  
    title = "Centrales nucléaires"  
  )
```

Ex : On peut isoler un type d'infrastructure pour construire une variable d'exposition spécifique.

Harmoniser les projections

Avant certaines opérations spatiales, les objets doivent avoir le même CRS.

```
st_crs(world_map)  
st_crs(power_plants)
```

Si nécessaire, on transforme la projection.

```
power_plants <- st_transform(power_plants, st_crs(world_map))
```

Avant une intersection, une distance ou une jointure spatiale, vérifier que les CRS sont compatibles.

Sélectionner une zone d'étude

On peut restreindre l'analyse à une zone géographique.

```
europe_map <- world_map %>%  
  filter(continent == "Europe")
```

On peut ensuite garder les points situés dans cette zone.

```
power_plants_europe <- power_plants %>%  
  st_filter(europe_map)
```

`st_filter()` permet de garder les objets qui intersectent une couche spatiale.

Points dans polygones

Une opération classique consiste à identifier dans quel polygone se trouve chaque point.

```
plants_with_country <- power_plants %>%  
  st_join(world_map)
```

Cette opération ajoute aux points les informations du polygone dans lequel ils se trouvent.

Exemple :

- une centrale reçoit le code du pays où elle est localisée ;
- une école reçoit le nom de la commune ;
- un ménage reçoit l'identifiant du district.

Compter des points dans des polygones

On peut compter le nombre de points dans chaque pays ou région.

```
points_in_country <- st_intersects(world_map, power_plants)

world_map$n_power_plants <- lengths(points_in_country)
```

La variable `n_power_plants` indique le nombre de centrales dans chaque pays.

Ex : Construire le nombre d'infrastructures disponibles dans une unité administrative.

Cartographier un comptage

Une fois le nombre de points calculé, on peut produire une carte.

```
ggplot(world_map) +  
  geom_sf(  
    aes(fill = n_power_plants),  
    color = "grey80",  
    size = 0.05  
  ) +  
  scale_fill_viridis_c(  
    na.value = "grey90"  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Nombre de centrales électriques par pays",  
    fill = "Nombre"  
  )
```

Attention : Un nombre total dépend souvent de la taille du pays : une densité peut parfois être plus informative.

Calculer une densité

Pour comparer des pays de tailles différentes, on peut calculer une densité.

```
world_map <- world_map %>%  
  st_transform(3857) %>%  
  mutate(  
    area_km2 = as.numeric(st_area(geometry)) / 1e6,  
    density_power_plants = n_power_plants / area_km2  
  )
```

Cette variable mesure le nombre de centrales par km².

Passer d'un nombre total à une densité peut changer fortement l'interprétation.

Calculer des centroïdes

Pour calculer des distances entre pays, on peut utiliser leurs centroïdes.

```
world_centroids <- world_map %>%  
  st_centroid()
```

Les centroïdes sont des points représentant approximativement les polygones.

Ils peuvent être utilisés pour :

- calculer des distances entre pays ;
- placer des labels ;
- simplifier des géométries complexes.

Attention : Le centroïde est une approximation : il peut être peu pertinent pour certains pays très étendus ou discontinus.

Calculer une distance à une infrastructure

On peut calculer la distance entre des pays et une infrastructure.

Exemple : distance entre les centroïdes des pays et les centrales nucléaires.

```
world_centroids <- world_centroids %>%  
  st_transform(3857)  
  
nuclear_plants <- nuclear_plants %>%  
  st_transform(3857)  
  
dist_to_nuclear <- st_distance(world_centroids, nuclear_plants)
```

Le résultat est une matrice de distances.

Attention : Pour interpréter une distance en mètres, il faut travailler dans une projection adaptée.

Distance minimale

À partir de la matrice de distances, on peut calculer la distance à l'infrastructure la plus proche.

```
world_centroids$dist_nearest_nuclear_km <- apply(
  dist_to_nuclear,
  1,
  min
) / 1000
```

Cette variable mesure, pour chaque pays, la distance au point le plus proche.

Ex : Distance à l'école la plus proche, au centre de santé le plus proche, à la route la plus proche, à une frontière ou à une antenne.

Créer une zone tampon autour des points

On peut créer une zone autour de chaque point avec `st_buffer()`.

```
nuclear_buffer_50km <- nuclear_plants %>%  
  st_buffer(dist = 50000)
```

Ici, on crée une zone de 50 km autour de chaque centrale nucléaire.

Usage : Les buffers permettent de définir une zone d'exposition autour d'une infrastructure ou d'un événement.

Identifier les zones exposées

On peut identifier les polygones qui intersectent une zone tampon.

```
world_map$near_nuclear_50km <- lengths(  
  st_intersects(world_map, nuclear_buffer_50km)  
) > 0
```

Cette variable vaut TRUE si le pays intersecte au moins une zone de 50 km autour d'une centrale.

Ex : Créer une variable binaire d'exposition à une infrastructure ou à un traitement.

Cartographier une exposition

On peut représenter la variable d'exposition sur une carte.

```
ggplot(world_map) +  
  geom_sf(  
    aes(fill = near_nuclear_50km),  
    color = "grey80",  
    size = 0.05  
  ) +  
  theme_minimal() +  
  labs(  
    title = "Pays situés à proximité d'une centrale nucléaire",  
    fill = "À moins de 50 km"  
  )
```

La carte distingue les territoires exposés et non exposés selon une règle spatiale explicite.

Différence entre jointure et intersection

Il faut distinguer deux opérations.

- **Jointure attributaire** : on utilise une clé commune.
 - Exemple : joindre par code pays.
- **Jointure spatiale** : on utilise la position géographique.
 - Exemple : attribuer un pays à chaque point selon sa localisation.

Exemples de variables empiriques

Les opérations spatiales permettent de construire de nombreuses variables.

- distance à la route la plus proche ;
- distance à la capitale ;
- nombre d'écoles dans une commune ;
- exposition à une infrastructure dans un rayon donné ;
- appartenance à une zone traitée ;
- distance à une frontière ;
- densité d'événements autour d'un lieu ;
- nombre de conflits dans une cellule.

Les erreurs classiques

Quelques erreurs sont fréquentes lorsque l'on manipule des points et des distances.

- calculer des distances avec des coordonnées en degrés ;
- oublier de transformer les CRS ;
- compter des points sans vérifier les frontières utilisées ;
- confondre distance au centroïde et distance réelle ;
- créer un buffer avec une mauvaise unité ;
- interpréter une exposition spatiale comme un effet causal.

Introduction aux rasters

Introduction aux rasters

Dans ce bloc, nous introduisons un autre grand type de données spatiales : les données raster.

Nous verrons comment :

- comprendre la structure d'un raster ;
- lire un fichier raster dans R ;
- afficher un raster ;
- manipuler une grille de pixels ;
- extraire des valeurs raster vers des points ou des polygones ;
- utiliser ces données pour construire des variables empiriques.

Un raster est une grille spatiale : chaque pixel contient une valeur.

Qu'est-ce qu'un raster ?

Un raster est une image géoréférencée.

Il est composé de cellules ou de pixels.

Chaque pixel contient une valeur :

- température ;
- précipitations ;
- altitude ;
- luminosité nocturne ;
- couverture forestière ;
- pollution ;
- accessibilité.

Ex : Un raster climatique peut indiquer la température moyenne dans chaque pixel d'une grille mondiale.

Pourquoi les rasters sont utiles en économie ?

Les rasters permettent de mesurer des phénomènes continus dans l'espace.

Ils sont très utilisés pour étudier :

- les effets du climat sur l'agriculture ;
- l'impact des sécheresses sur les revenus ;
- la relation entre activité économique et luminosité nocturne ;
- l'exposition à la pollution ;
- l'accessibilité aux marchés ;
- l'occupation des sols.

Les rasters permettent d'associer une information environnementale ou géographique à des unités économiques.

Vecteur ou raster ?

Les données vectorielles et raster ne décrivent pas l'espace de la même manière.

- **Vecteur** : objets géographiques précis.
 - écoles, routes, communes, pays.
- **Raster** : grille régulière de pixels.
 - température, pluie, altitude, night lights.

Exemple de question empirique

Supposons que l'on étudie l'effet de la chaleur sur les revenus agricoles.

On peut combiner :

- une base de ménages ou de communes ;
- une carte des limites administratives ;
- un raster de température ;
- éventuellement un raster de précipitations.

Objectif :

raster climatique $\rightarrow$ exposition locale $\rightarrow$ variable empirique

Ex : Calculer la température moyenne observée dans chaque commune, puis l'utiliser dans une régression.

Le package terra

Pour manipuler des rasters dans R, on utilise aujourd'hui souvent le package terra.

```
library(terra)
```

terra permet notamment de :

- lire des fichiers raster ;
- afficher des rasters ;
- reprojeter des rasters ;
- découper des rasters ;
- extraire des valeurs vers des points ou polygones ;
- faire des calculs sur les pixels.

Beaucoup d'anciens codes utilisent raster. Pour les nouveaux projets, on privilégie souvent terra.

Lire un raster

On lit un fichier raster avec la fonction `rast()`.

```
temp_raster <- rast("data/raw/temperature.tif")
```

On peut ensuite inspecter l'objet.

```
temp_raster
```

```
crs(temp_raster)
```

```
plot(temp_raster)
```

`rast()` est l'équivalent raster de `st_read()` pour les données vectorielles.

Comprendre un raster

Un raster contient plusieurs informations importantes.

- le nombre de lignes et de colonnes ;
- la résolution spatiale ;
- l'étendue géographique ;
- le système de coordonnées ;
- le nombre de couches ;
- les valeurs des pixels.

```
nrow(temp_raster)
ncol(temp_raster)
res(temp_raster)
ext(temp_raster)
crs(temp_raster)
```

Avant d'utiliser un raster, vérifier sa résolution, son étendue et son CRS.

Raster à une couche ou plusieurs couches

Un raster peut contenir une seule couche ou plusieurs couches.

Exemples :

- une couche : température moyenne annuelle ;
- douze couches : température moyenne par mois ;
- plusieurs années : night lights par année ;
- plusieurs variables : température, pluie, altitude.

On peut afficher le nom des couches :

```
names(temp_raster)
```

Et sélectionner une couche :

```
temp_may <- temp_raster[[5]]
```

Ex : Si chaque couche correspond à un mois, `[[5]]` peut représenter le mois de mai.

Afficher un raster

On peut afficher rapidement un raster avec `plot()`.

```
plot(temp_may)
```

Pour une carte plus personnalisée, on peut convertir le raster en tableau.

```
temp_may_df <- as.data.frame(  
  temp_may,  
  xy = TRUE,  
  na.rm = TRUE  
)
```

Pour explorer rapidement, `plot()` suffit. Pour produire une figure personnalisée, on peut passer par `ggplot2`.

Cartographier un raster avec ggplot2

Après conversion en tableau, on peut utiliser ggplot2.

```
ggplot(temp_may_df, aes(x = x, y = y)) +  
  geom_raster(aes(fill = temp_may)) +  
  coord_equal() +  
  theme_minimal() +  
  labs(  
    title = "Température moyenne en mai",  
    x = "Longitude",  
    y = "Latitude",  
    fill = "Température"  
  )
```

`geom_raster()` permet de représenter une grille de pixels avec ggplot2.

Découper un raster

Souvent, on ne veut garder qu'une zone d'étude.

Exemple : découper un raster sur l'Europe.

```
europe_map <- world_map %>%  
  filter(continent == "Europe")  
  
temp_europe <- crop(temp_raster, vect(europe_map))  
temp_europe <- mask(temp_europe, vect(europe_map))
```

- `crop()` réduit l'étendue rectangulaire du raster ;
- `mask()` conserve uniquement la zone exacte du polygone.

Bonne pratique : Découper un raster permet de réduire la taille des données et de travailler uniquement sur la zone utile.

Harmoniser les projections

Comme pour les vecteurs, les rasters ont un CRS.

Avant de combiner raster et vecteur, il faut vérifier leurs systèmes de coordonnées.

```
crs(temp_raster)
st_crs(world_map)
```

Si nécessaire, on peut reprojeter le raster.

```
temp_raster_proj <- project(
  temp_raster,
  "EPSG:4326"
)
```

Attention : Reprojecter un raster peut être coûteux et modifier les valeurs par interpolation.

Extraire une valeur raster vers des points

Si l'on dispose de points, on peut extraire la valeur du pixel correspondant.

```
points_temp <- terra::extract(  
  temp_raster,  
  vect(points_sf)  
)
```

On peut ensuite ajouter la valeur extraite à la base de points.

```
points_sf$temp <- points_temp[, 2]
```

Ex : Associer à chaque école, ménage ou entreprise la température observée à sa localisation.

Extraire une moyenne vers des polygones

Si l'on dispose de polygones, on peut calculer une moyenne des pixels dans chaque zone.

Avec le package `exactextractr` :

```
library(exactextractr)

world_map$mean_temp <- exact_extract(
  temp_raster,
  world_map,
  "mean"
)
```

Cette commande calcule la température moyenne dans chaque polygone.

Ex : Calculer la pluie moyenne par commune, la température moyenne par district ou les night lights moyens par région.

Construire une base d'analyse avec un raster

L'objectif est souvent d'ajouter une variable raster à une base statistique.

Exemple de workflow :

raster → extraction par polygone → nouvelle variable → base d'analyse

```
communes$mean_temp <- exact_extract(  
  temp_raster,  
  communes,  
  "mean"  
)  
  
analysis_data <- communes %>%  
  st_drop_geometry() %>%  
  select(code_commune, mean_temp, population, income)
```

Exemple : luminosité nocturne

Les données de luminosité nocturne sont souvent utilisées comme proxy d'activité économique locale.

Elles permettent d'étudier :

- la croissance locale ;
- l'activité économique dans les zones sans statistiques fiables ;
- les effets de conflits ou catastrophes ;
- l'électrification ;
- l'urbanisation.

Workflow typique :

raster night lights → moyenne par unité géographique → panel spatial

Attention : Les night lights sont utiles, mais ce sont des proxies : leur interprétation doit être prudente.

Exemple : climat et agriculture

Les rasters climatiques permettent de mesurer l'exposition aux conditions météorologiques.

Exemples de variables :

- température moyenne ;
- nombre de jours très chauds ;
- cumul annuel de précipitations ;
- anomalie de pluie ;
- indice de sécheresse.

Ces variables peuvent être reliées à :

- production agricole ;
- revenus ;
- migrations ;
- santé ;

Les erreurs classiques avec les rasters

Quelques erreurs sont fréquentes.

- oublier de vérifier la résolution du raster ;
- mélanger des rasters avec des CRS différents ;
- interpréter un pixel comme une observation individuelle ;
- extraire une moyenne sans vérifier la zone couverte ;
- oublier les valeurs manquantes ;
- comparer des rasters de résolution différente sans précaution ;
- oublier l'unité de la variable.